

Attention Is All You Need

A First-Principles Complete Course

From absolute beginner → full understanding, explanation,
mental simulation, and miniature PyTorch implementation

Based on Vaswani et al., 2017 (arXiv:1706.03762)

Parts 0–20 · Intuition → Mathematics → Computation → Architecture

Teaching rule: for every component we ask — What problem does this solve? What happens to the tensor? Why is this operation necessary? What fails if we remove it? Where does the paper state it?

Running examples throughout:

English → **French**: “I love cats” → “J’aime les chats”

Pronoun: “The animal didn’t cross the road because it was tired.”

Architecture note: this course reconstructs the original 2017 Post-LN Transformer and explicitly contrasts it with modern Pre-LN variants.

Table of Contents

- Part 0 — How to Read This Paper
- Part 1 — Minimum Prerequisites (the vital 20%)
- Part 2 — Why Attention Was Needed
- Part 3 — Attention From Zero (full numerical walkthrough)
- Part 4 — Self-Attention (contextualization + cost)
- Part 5 — Multi-Head Attention (subspaces + shapes)
- Part 6 — Positional Encoding (order without recurrence)
- Part 7 — The Feed-Forward Network
- Part 8 — Residual Connections + Layer Normalization (Post-LN vs Pre-LN)
- Part 9 — Build the Encoder From Scratch
- Part 10 — The Decoder (masking + cross-attention)
- Part 11 — Complete Transformer Data Flow
- Part 12 — Training the Transformer
- Part 13 — Inference / Generation
- Part 14 — The Paper's Equations (decoded)
- Part 15 — Original 2017 Architecture Specs
- Part 16 — Complexity and Why Transformers Scale
- Part 17 — Reading the Results Section
- Part 18 — Limitations of the Original Transformer
- Part 19 — Implement a Miniature Transformer (PyTorch)
- Part 20 — Final Master Understanding (quiz + answer key)

This PDF is self-contained. Read in order. Do not skip numerical examples — they are where understanding becomes real. Parts 3, 4, 5, 6, 10, 11, 12, and 19 are intentionally the densest.

PART 0 — How to Read This Paper

0.1 What is a research paper?

A research paper is a compressed scientific argument. It is not a textbook and not a blog tutorial. Authors write for other researchers who already share vocabulary, notation, and prior results. That is why a first encounter with “Attention Is All You Need” feels dense: many sentences assume you already know seq2seq, attention, and BLEU.

Almost every ML paper has this skeleton:

- **Problem:** something is limited, expensive, inaccurate, or hard to scale
- **Idea:** a new method, architecture, training procedure, or theory
- **Method:** precise definition of the model and how it is trained
- **Evidence:** experiments showing it works (often better than baselines)
- **Limits:** what remains unsolved (sometimes understated)

Your first goal is not to understand every sentence. Your first goal is to extract: problem → idea → method → evidence.

0.2 How to read a machine-learning paper (multiple passes)

Professionals almost never read linearly once. Use passes with different jobs.

- **Pass 1 — Map:** title, abstract, first page of intro, all figures, conclusion. What claim? What is Figure 1? What datasets?
- **Pass 2 — Architecture:** read §3 while staring at Figure 1. Trace input → process → output.
- **Pass 3 — Equations:** for each equation list symbols, shapes, and the actual computation. Recompute with tiny numbers.
- **Pass 4 — Experiments:** datasets, metrics, baselines, ablations, training cost.
- **Pass 5 — Limitations:** what they did not solve; what assumptions remain; what later work changed.

Common mistake: treating every sentence as equally important. Better habit: classify each paragraph as motivation / architecture / equation / experiment / claim.

0.3 What problem are the authors solving?

In 2017, strong sequence models (especially machine translation) relied on recurrence (RNNs/LSTMs) and sometimes convolutions. Recurrence processes tokens one-by-one: hidden state h_t waits for h_{t-1} . That sequential dependence makes training hard to parallelize across time and makes long-range dependencies travel through many steps.

Core research question: Can we build a strong sequence transduction model that relies entirely on attention — with no recurrence and no convolution? Their answer is the Transformer.

0.4 What “contribution” means

A contribution is the new reusable thing given to the field. For this paper: an encoder–decoder architecture based solely on attention (self-attention + multi-head attention + cross-attention), with strong translation quality and better training parallelization than recurrent models.

*Later models (BERT, GPT, ViT, LLaMA, ...) are **impact** — not this paper’s claimed 2017 contribution. Stay loyal to 2017 when reconstructing the architecture (Post-LN, sinusoidal PE, encoder–decoder MT).*

0.5 Distinguishing paper pieces

Piece	Answers	Where in paper
-------	---------	----------------

Motivation	Why it matters	§1 Introduction
Architecture	What the model is	§3 Model + Fig. 1
Equations	Exact computation	§3.2–3.5
Training	How optimized	§5 Training
Results	Did it work?	§6 Results (BLEU)
Ablations	What parts matter?	§5.3 / tables
Limitations	Still unsolved	implied + later work

0.6 Mental map of the paper

[PROBLEM] Recurrent MT is sequential → hard parallelization, hard long-range deps
 [IDEA] Attention only → Transformer encoder-decoder
 [CORE] Scaled Dot-Product Attention → Multi-Head → Self / Cross / Masked
 [SUPPORT] Embeddings + Positional Encoding; FFN; Add & Norm (Post-LN); N=6 stacks
 [TRAIN] Teacher forcing, cross-entropy, Adam+schedule, dropout, label smoothing
 [EVIDENCE] WMT BLEU, speed/params, ablations
 [GOAL] Mentally simulate tokens → probabilities and explain WHY each piece exists

0.7 Three buckets you must never mix

- **Architecture:** what tensors exist and how they transform (Fig. 1, equations).
- **Training procedure:** loss, optimizer, schedule, teacher forcing, label smoothing.
- **Implementation details:** dropout rates, batching tricks, beam search hyperparameters.

Separate forever: architecture vs training procedure vs implementation details.

0.8 How this course maps onto the paper

- Parts 1–2 build prerequisites and historical motivation (§1–2).
- Parts 3–8 unpack every block inside Fig. 1 (§3.2–3.5).
- Parts 9–11 assemble encoder, decoder, and full dataflow (§3.1).
- Parts 12–13 cover training and inference (§5).
- Parts 14–17 decode equations, specs, complexity, and results (§4–6).
- Parts 18–20 limitations, code, and mastery check.

PART 1 — Minimum Prerequisites (Only the Vital 20%)

We will not dump an entire deep-learning course. These are the concepts you need for 80–90% of the paper. If a later part mentions a shape you do not understand, return here.

1.1 Vectors

A vector is an ordered list of numbers. In Transformers, almost everything is a vector: a token embedding, a query, a key, a value, an attention output, an FFN hidden state.

```
v = [1.0, 0.5, -0.2]    # shape: (3,)    a 3-dimensional vector
```

Intuition: each dimension is one learned feature axis. You do not need to interpret each axis by hand. What matters is geometry: directions, lengths, and angles between vectors.

1.2 Matrices and shapes

A matrix is a rectangle of numbers: rows $\times$ columns. Notation: $X \in \mathbb{R}^{(T \times d)}$ means X has T rows and d columns, with real numbers. In this course, **one row = one token's representation**.

```
X ∈ ℝ^(T × d)
T = sequence length (number of tokens)
d = model dimension (size of each token vector)
one ROW = one token's representation
one COL = one feature across all tokens
```

```
Example: 3 tokens ("I", "love", "cats"), d=4
X = [[ 0.1, 0.2, 0.0, 0.4], # "I"
      [ 0.5, 0.1, 0.3, 0.0], # "love"
      [-0.2, 0.6, 0.1, 0.2]] # "cats"
shape(X) = (3, 4)
```

1.3 Matrix multiplication

If A is $(m \times n)$ and B is $(n \times p)$, then AB is $(m \times p)$. The inner dimensions must match (both n). Each entry $C[i,j]$ is the dot product of row i of A with column j of B .

```
A (2×3) @ B (3×2) → C (2×2)
```

Why Transformers care: multiplying by a weight matrix W is how we linearly transform vectors (e.g., turn embeddings into Queries / Keys / Values, or project to vocabulary logits).

```
Shape drill:
X (T×d) @ W (d×d_k) → (T×d_k)    # project every token
Q (T×d_k) @ K (d_k×T) → (T×T)    # all-pairs scores
```

1.4 Dot products as similarity

The dot product of two equal-length vectors is the sum of elementwise products. Large positive $\approx$ similar direction; near zero $\approx$ unrelated; negative $\approx$ opposite.

```
q = [1, 2, 0]
k = [1, 0, 3]
q · k = 1*1 + 2*0 + 0*3 = 1
```

In attention, `score(query, key)` starts as a dot product:
"how well does this query match this key?"

1.5 Softmax and probability

Softmax turns a list of real scores into a probability distribution: all values positive and sum to 1.

$$\text{softmax}(z)_i = \exp(z_i) / \sum_j \exp(z_j)$$

```
scores = [2.0, 1.0, 0.1]
exp     = [7.389, 2.718, 1.105]
sum     = 11.212
probs   = [0.659, 0.242, 0.099] # attention weights / next-token probs
```

In attention, softmax tells each token: “how much weight should I give each other token?” In the final layer, softmax over vocabulary tells: “how likely is each next word?”

1.6 Neural networks (minimum)

A neural network layer is usually: linear transform → nonlinearity → (maybe) another linear transform. Learned parameters (weights/biases) are adjusted by gradient descent so predictions improve.

```
y = activation(x W + b) # e.g., ReLU = max(0, .)
```

1.7 Embeddings

Computers do not understand words. An embedding table maps each token ID to a learned vector of size `d_model`. Similar words often end up with similar vectors after training — but embeddings alone are not contextual. “Bank” in “river bank” and “bank account” start with the same embedding until context is mixed in by attention.

```
vocab: {"I":0, "love":1, "cats":2, "<BOS>":3, "J'aime":4, "les":5, "chats":6, "<EOS>":7}
ids:   [0, 1, 2]
E = EmbeddingLookup(ids) # shape (3, d_model)
```

1.8 Sequence-to-sequence + encoder–decoder

Sequence-to-sequence (seq2seq) means: input sequence → output sequence of possibly different length (translation, summarization). Classic pattern:

```
ENCODER: read source sentence, build contextual representations (memory)
DECODER: generate target sentence token by token, using encoder memory
```

```
Example:
source: I love cats
target: J'aime les chats
```

1.9 RNNs/LSTMs at conceptual level

An RNN reads tokens left-to-right, updating a hidden state. LSTMs improve memory of longer signals, but the core constraint remains: step t waits for step $t-1$. That sequential dependence is the bottleneck the Transformer attacks.

```
RNN:
h1 = f(x1, h0)
h2 = f(x2, h1) # must wait for h1
h3 = f(x3, h2) # must wait for h2
Cannot fully parallelize across time on a GPU.
```

1.10 Loss, gradients, and “learning”

Training compares the model’s predicted token probabilities to the true next tokens (cross-entropy). Backpropagation computes how every weight contributed to the error; an optimizer (Adam) nudges weights to reduce that error. You do not need to derive gradients by hand to understand the architecture — but you must know that every matmul, softmax, and LayerNorm in the Transformer is differentiable.

Checkpoint: if $X \in \mathbb{R}^{(T \times d)}$, you should instantly know T =tokens, d =features per token, rows=token vectors.

1.11 Shape drills (do these mentally)

```
Drill A: X (5×64), W (64×32) → ?      Answer: (5×32)
Drill B: Q (5×32), K (5×32) → QK■ ?   Answer: (5×5)
Drill C: A (5×5), V (5×32) → ?        Answer: (5×32)
Drill D: concat 8 heads of (5×64) → ? Answer: (5×512)
```

Drill E: logits (5×10000) softmax on last dim → each row sums to 1

1.12 Notation quick reference

Symbol	Meaning
T / n	sequence length
d_{model} / d	model width
d_k, d_v	key/query and value widths
h	number of heads
d_{ff}	FFN hidden width
N	number of layers
W_Q, W_K, W_V, W_O	attention projection matrices

PART 2 — Why Attention Was Needed

2.1 The story arc

RNN → LSTM → still sequential → long-range pain
→ Attention (decoder looks selectively at encoder states)
→ Self-Attention (tokens attend to tokens in same sequence)
→ Transformer (attention is the whole architecture; no RNN, no CNN)

2.2 Why recurrence is sequential and costly

In an RNN, computing hidden state h_t requires h_{t-1} . GPUs love parallel matrix multiplies, but recurrence forces a chain through time. For long sentences, training cannot fully parallelize across positions. Wall-clock training time grows roughly with sequence length even when you have plenty of FLOPs available.

2.3 Long-range dependencies (pronoun example)

Example: “The animal didn’t cross the road because it was tired.” To resolve “it”, the model must relate a pronoun to a noun several tokens away. In deep recurrence, signals travel many steps and can weaken or be overwritten. Attention creates a direct link in one step: the query for “it” can put weight directly on the key for “animal”.

Distance in sentence:
animal ----(several tokens)---- it

RNN path length $\approx$ number of steps between them
Attention path length = 1 (direct score in QK)

2.4 Why convolutions also struggle for long range

Convolutions see local neighborhoods. Capturing long distance requires many layers or huge kernels, increasing path length between distant tokens. Attention connects any pair directly (path length $O(1)$). This is Table 1’s qualitative argument in the paper.

2.5 What attention buys

- **Selective focus:** weight relevant tokens more, ignore others softly
- **Direct long-range links:** any token can influence any other in one layer
- **Parallelism:** all token pairs can be scored with matrix multiplies at once

“Parallelization” means: during training, the model can compute attention for the whole sequence simultaneously instead of waiting token-by-token like an RNN.

2.6 From generic attention to self-attention

Earlier seq2seq (Bahdanau et al.) used attention from decoder steps to encoder states — a huge win, but the backbone was still recurrent. Self-attention goes further: within one sequence, every token builds a new representation by attending over the same sequence. That is the Transformer’s central move: make attention the workhorse for representation learning, not just an add-on alignment module.

2.7 What would fail without this shift?

- Keep RNNs → keep sequential training bottleneck
- Keep only local convolutions → keep long path lengths
- Keep attention only as decoder→encoder add-on → miss deep contextualization inside each side

Paper connection: §1–2 motivate replacing recurrence/convolution with attention; §3 defines the model; §4 argues complexity / path length; §6 shows BLEU and speed.

PART 3 — Attention From Zero

This is the heart of the paper. Master this section and the rest becomes assembly. We will compute every intermediate matrix by hand.

3.1 The problem attention solves

Given a question (query), search a set of items (keys), and return a blend of their contents (values), weighted by how well keys match the query. This is a soft, differentiable dictionary lookup.

3.2 Analogy (then we leave the analogy)

Library analogy: Query = what you want; Keys = catalog labels on books; Values = book contents. You compare query to keys, get relevance weights, then read a weighted mixture of values. In neural attention, keys and values are vectors, and comparison is a scaled dot product. After this subsection we speak only in tensors.

3.3 Q, K, V without heavy math

- **Query (Q):** “What am I looking for?” — one vector per token asking the question
- **Key (K):** “What do I contain / advertise?” — one vector per token being matched against
- **Value (V):** “What information do I pass if selected?” — content to mix

Misconception to avoid: Q/K/V are not three different sentences. In self-attention they are three different linear projections of the same token representations. In cross-attention, Q comes from the decoder while K and V come from the encoder.

3.4 The equation

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}(\mathbf{Q} \mathbf{K}^\top / \sqrt{d_k}) \mathbf{V}$$

Symbol and shape dictionary (batch ignored):

- $\mathbf{Q} \in \mathbb{R}^{T_q \times d_k}$ — queries
- $\mathbf{K} \in \mathbb{R}^{T_k \times d_k}$ — keys
- $\mathbf{V} \in \mathbb{R}^{T_v \times d_v}$ — values (often $d_v = d_k$)
- $\mathbf{K}^\top \in \mathbb{R}^{d_k \times T_k}$ — transpose of K
- $\mathbf{Q} \mathbf{K}^\top \in \mathbb{R}^{T_q \times T_k}$ — raw similarity scores (every query vs every key)
- $\sqrt{d_k}$ — scale factor
- $\mathbf{A} = \text{softmax}(\mathbf{Q} \mathbf{K}^\top / \sqrt{d_k}) \in \mathbb{R}^{T_q \times T_k}$ — attention weights (each row sums to 1)
- $\text{output} = \mathbf{A} \mathbf{V} \in \mathbb{R}^{T_q \times d_v}$ — weighted sum of values for each query

3.5 Why divide by $\sqrt{d_k}$?

If Q and K have independent components with variance ~ 1 , the dot product has variance $\sim d_k$. For large d_k (e.g., 64), scores become large in magnitude $\rightarrow$ softmax becomes extremely peaky (almost one-hot) $\rightarrow$ gradients through softmax vanish. Scaling by $\sqrt{d_k}$ keeps score variance ~ 1 , so softmax stays informative and trainable.

Toy illustration of saturation:

```
logits = [10, 0, 0]      → softmax ≈ [1.000, 0.000, 0.000] (dead gradients for losers)
logits = [1.0, 0, 0]     → softmax ≈ [0.576, 0.212, 0.212] (usable gradients)
```

Scaling prevents the first regime when d_k is large.

Paper statement (§3.2.1) is brief; the underlying reason is softmax saturation under large-variance logits.

3.6 Complete numerical example — every intermediate matrix

Setup: 3 tokens, $d_k = d_v = 2$. We invent Q, K, V directly (as if projections already happened). Think of the three rows as toy stand-ins for “I”, “love”, “cats”.

Token rows: 0="I", 1="love", 2="cats"

```
Q = [[1.0, 0.0],      # query for "I"
      [0.0, 1.0],      # query for "love"
      [1.0, 1.0]]      # query for "cats"      shape (3,2)
```

```
K = [[1.0, 2.0],      # key for "I"
      [0.0, 1.0],      # key for "love"
      [1.0, 0.0]]      # key for "cats"      shape (3,2)
```

```
V = [[1.0, 0.0],      # value for "I"
      [0.0, 1.0],      # value for "love"
      [1.0, 1.0]]      # value for "cats"      shape (3,2)
```

Step A — Transpose K

```
KT = [[1.0, 0.0, 1.0],
        [2.0, 1.0, 0.0]]      shape (2,3)
```

Step B — Raw scores $S = Q K^T$

```
S[i,j] = dot(Q[i], K[j])
```

```
S[0,0] = 1*1 + 0*2 = 1
S[0,1] = 1*0 + 0*1 = 0
S[0,2] = 1*1 + 0*0 = 1
```

```
S[1,0] = 0*1 + 1*2 = 2
S[1,1] = 0*0 + 1*1 = 1
S[1,2] = 0*1 + 1*0 = 0
```

```
S[2,0] = 1*1 + 1*2 = 3
S[2,1] = 1*0 + 1*1 = 1
S[2,2] = 1*1 + 1*0 = 1
```

```
S = QKT = [[1, 0, 1],
             [2, 1, 0],
             [3, 1, 1]]      shape (3,3)
```

Step C — Scale by $\sqrt{d_k}$

```
 $\sqrt{d_k} = \sqrt{2} \approx 1.41421356$ 
scaled = S /  $\sqrt{2}$ 
```

```
scaled  $\approx$  [[0.7071, 0.0000, 0.7071],
             [1.4142, 0.7071, 0.0000],
             [2.1213, 0.7071, 0.7071]]
```

Step D — Softmax each row $\rightarrow$ attention matrix A

```
Row 0: z=[0.7071, 0.0000, 0.7071]
exp=[2.0287, 1.0000, 2.0287]; sum=5.0575
A[0]=[0.4011, 0.1977, 0.4011]
```

```
Row 1: z=[1.4142, 0.7071, 0.0000]
exp=[4.1133, 2.0287, 1.0000]; sum=7.1420
A[1]=[0.5759, 0.2841, 0.1400]
```

```
Row 2: z=[2.1213, 0.7071, 0.7071]
exp=[8.3431, 2.0287, 2.0287]; sum=12.4005
A[2]=[0.6728, 0.1636, 0.1636]
```

```
A  $\approx$  [[0.401, 0.198, 0.401],
        [0.576, 0.284, 0.140],
        [0.673, 0.164, 0.164]]
```

Check: each row ≈ 1.0 . Cell $A[i,j]$ = how much token i attends to token j .

Step E — Output = A V (weighted mix of values)

```
out[i] = A[i,0]*V[0] + A[i,1]*V[1] + A[i,2]*V[2]

out0 = 0.401*[1,0] + 0.198*[0,1] + 0.401*[1,1]
      = [0.401, 0] + [0, 0.198] + [0.401, 0.401]
      = [0.802, 0.599]

out1 = 0.576*[1,0] + 0.284*[0,1] + 0.140*[1,1]
      = [0.576, 0] + [0, 0.284] + [0.140, 0.140]
      = [0.716, 0.424]

out2 = 0.673*[1,0] + 0.164*[0,1] + 0.164*[1,1]
      = [0.673, 0] + [0, 0.164] + [0.164, 0.164]
      = [0.837, 0.328]

AttentionOutput ≈ [[0.802, 0.599],
                  [0.716, 0.424],
                  [0.837, 0.328]]      shape (3,2)
```

3.7 Trace diagram

```
Input projections → Q, K, V
    ↓
scores S = Q KT          (Tq × Tk)
    ↓
scaled = S / √dk
    ↓
A = softmax_row(scaled) (attention weights)
    ↓
Out = A V                (Tq × dv)
```

3.8 What fails if a piece is removed?

- **No softmax:** weights are not a distribution; mixing is unnormalized and unstable.
- **No scaling (large d_k):** sharp softmax, vanishing gradients, hard training.
- **No V:** you have similarities but no content to pass — matching without readout.
- **No Q or K:** you cannot form content-based addressing; collapses toward fixed averaging.

3.9 Where it fits + paper connection

Scaled dot-product attention is the atomic operation inside every Multi-Head Attention block: encoder self-attention, decoder masked self-attention, and decoder–encoder cross-attention. Paper: §3.2.1 Scaled Dot-Product Attention; Figure 2 (left).

3.10 Second worked example — pronoun toy (T=4, d_k=2)

Tokens: animal, road, it, tired. We hand-craft Q so that “it” matches “animal”.

```
Q = [[0,1], [1,0], [1,1], [0,1]] # animal, road, it, tired
K = [[1,1], [1,0], [0,1], [0,1]]
V = [[1,0], [0,1], [0,0], [1,1]]
```

```
S = QKT =
[[1,0,1,1],
 [1,1,0,0],
 [2,1,1,1],
 [1,0,1,1]]
```

```
scaled = S/√2
Row "it" (index 2): z≈[1.414, 0.707, 0.707, 0.707]
A[it] ≈ [0.42, 0.19, 0.19, 0.19] # largest on animal
```

out_it mixes values $\approx$ mostly animal's [1,0] (+ some tired)
→ contextualized "it" carries animal information – coreference in one matmul.

This is the computational form of the linguistic story from Part 2: direct long-range links.

3.11 Batch and head dimensions (preview)

In code you usually see:

```
Q: (batch, heads, T_q, d_k)
K: (batch, heads, T_k, d_k)
V: (batch, heads, T_k, d_v)
scores: (batch, heads, T_q, T_k)
```

The math is identical per (batch, head) slice – Part 3 is that slice.
Multi-head (Part 5) runs Part 3 in several subspaces, then concatenates.

3.12 Checklist: can you recompute attention cold?

- Write Q,K,V shapes from T and d_k/d_v
- Compute one score by hand as a dot product
- Scale, softmax one row, check sum $\approx$ 1
- Form one output row as weighted sum of V rows
- Name where this sits in Fig. 1

PART 4 — Self-Attention

4.1 What makes it “self”?

In self-attention, Q, K, and V all come from the **same** sequence (after linear projections). Each token updates itself by looking at every token including itself. This is how a static embedding becomes a contextualized representation.

4.2 From X to Q, K, V

```
X ∈ R^(T × d_model)           # token vectors (after emb+PE, or previous layer)

W_Q ∈ R^(d_model × d_k)
W_K ∈ R^(d_model × d_k)
W_V ∈ R^(d_model × d_v)

Q = X W_Q                      # (T × d_k)
K = X W_K                      # (T × d_k)
V = X W_V                      # (T × d_v)

A = softmax(Q K^T / √d_k)      # (T × T)
Y = A V                        # (T × d_v) contextualized (single-head)
```

4.3 Pronoun sentence — qualitative attention

Sentence: “The animal didn’t cross the road because it was tired.” For token “it”, a good attention pattern should put substantial mass on “animal” (the antecedent). Self-attention can learn such links because “it”’s query can match “animal”’s key.

Simplified attention row for query=“it” (illustrative, not trained weights):

```
tokens: The animal didn't cross road because it was tired
weights: 0.02 0.55 0.02 0.03 0.08 0.05 0.10 0.05 0.10
          ↑
        most mass on "animal"
```

4.4 Attention matrix visualization

Attention weights A (rows = query token, cols = key token):

	The	animal	road	it	tired
The	0.40	0.20	0.15	0.10	0.15
animal	0.10	0.50	0.10	0.10	0.20
road	0.10	0.15	0.45	0.10	0.20
it	0.05	0.55	0.10	0.15	0.15
tired	0.05	0.25	0.10	0.20	0.40

Cell (i,j) = how much token i attends to token j.
Each ROW sums to 1 (after softmax).

4.5 Tiny numerical self-attention with explicit W_Q, W_K, W_V

Now we start from X and projections — not from pre-made Q,K,V. T=3 tokens (“I”, “love”, “cats”), d_model=2, d_k=d_v=2 (single head toy).

```
X = [[1.0, 0.0], # I
      [0.0, 1.0], # love
      [1.0, 1.0]] # cats      (3×2)

W_Q = [[1, 0],
        [0, 1]]      # identity for clarity
W_K = [[1, 0],
        [0, 1]]
W_V = [[1, 0],
        [0, 1]]
```

Then $Q=X$, $K=X$, $V=X$.

$$S = QK^T = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 2 \end{bmatrix}$$
$$\text{scaled} = S/\sqrt{2} \approx \begin{bmatrix} 0.707 & 0.000 & 0.707 \\ 0.000 & 0.707 & 0.707 \\ 0.707 & 0.707 & 1.414 \end{bmatrix}$$
$$A \approx \begin{bmatrix} 0.401 & 0.198 & 0.401 \\ 0.212 & 0.394 & 0.394 \\ 0.274 & 0.274 & 0.452 \end{bmatrix}$$
$$Y = A V = A X \approx \begin{bmatrix} 0.802 & 0.599 \\ 0.606 & 0.788 \\ 0.726 & 0.726 \end{bmatrix}$$

Interpretation: each output row is a soft blend of the three input token vectors. "I" mixes mostly with itself and "cats"; "love" mixes with "love" and "cats"; etc.

4.6 Contextualized representations

After self-attention, a token's vector is no longer only its embedding. It is a mixture of values from related tokens — a contextualized representation. Stacking layers deepens this mixing: layer 1 mixes local/direct relations; deeper layers mix already-mixed signals (higher-order context).

4.7 Cost of self-attention

Self-attention compares all pairs $\rightarrow$ time/memory roughly $O(T^2 \cdot d)$ for sequence length T . This is the famous quadratic cost. Worth it for moderate T because of parallelization and short path length. For very long T , memory for the $(T \times T)$ score matrix becomes the bottleneck — motivating later sparse/linear attention.

4.8 What fails without self-attention?

- Without it inside the encoder: source tokens would not deeply contextualize each other before decoding.
- Without it inside the decoder (masked): target tokens could not condition on previous target tokens.
- Replacing with RNN: reintroduces sequential bottleneck the paper set out to remove.

Paper connection: §3.2. Self-attention is the content of the "Multi-Head Attention" boxes when $Q=K=V$ source.

4.9 Step-by-step: one encoder self-attention for "I love cats"

Goal: contextualize three English tokens before translation.

1. X ($3 \times d$) arrives (emb+PE or previous layer)
2. Project: $Q, K, V = X W_Q, X W_K, X W_V$
3. $S = QK^T$ (3×3): how every token scores every token
4. Scale / softmax $\rightarrow A$ (3×3)
5. $Y = A V$ ($3 \times d_v$): each token is now a mixture
6. (In multi-head: do this per head, concat, W^O)
7. Residual+norm with X (Part 8), then FFN (Part 7)

If "cats" should influence "love" (verb-object), $A[\text{love}, \text{cats}]$ rises after training.

4.10 Self-attention vs bag-of-embeddings

Average-pooling all embeddings is a fixed mixture. Self-attention is a **content-dependent** mixture: weights change with the query. That content dependence is why "bank" can mean river or finance depending on neighbors.

4.11 Attention pattern reading practice

Suppose after training, encoder A for "I love cats" looks like:

	I	love	cats
I	0.50	0.30	0.20
love	0.20	0.40	0.40
cats	0.15	0.25	0.60

Reading rows:

- "I" mostly self + some verb
- "love" balances self and object "cats"
- "cats" strongly self (object identity)

These numbers are illustrative; real heads differ and stack across layers.

PART 5 — Multi-Head Attention

5.1 Why one attention head is not enough

A single attention distribution per token is a compromise: one weighting must capture syntax, position, coreference, and semantics together. Multiple heads let the model attend in several subspaces at once — different learned projections create different notions of “similarity.”

Conceptual examples (not a claim that every head is cleanly interpretable in a trained model): Head1≈syntax adjacency, Head2≈positional patterns, Head3≈coreference/semantics. Real heads are messier; the architectural point is capacity for multiple relation types.

5.2 Formula

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$
$$\text{head}_i = \text{Attention}(Q W_i^Q, K W_i^K, V W_i^V)$$

5.3 Shapes in the Base model

Paper Base config:

$d_{\text{model}} = 512, h = 8 \rightarrow d_k = d_v = d_{\text{model}}/h = 64$

Input X: $(T \times 512)$

For each head $i = 1..8$:

$W_i^Q: (512 \times 64), Q_i = X W_i^Q \rightarrow (T \times 64)$

$W_i^K: (512 \times 64), K_i = X W_i^K \rightarrow (T \times 64)$

$W_i^V: (512 \times 64), V_i = X W_i^V \rightarrow (T \times 64)$

$\text{head}_i = \text{Attention}(Q_i, K_i, V_i) \rightarrow (T \times 64)$

Concat heads: $(T \times 512) \quad \# 8 \times 64 = 512$

$W^O: (512 \times 512)$

Output: $(T \times 512)$

Implementation trick: one big $\text{Linear}(d_{\text{model}} \rightarrow d_{\text{model}})$ for Q, then reshape to (T, h, d_k) . Same for K, V. Mathematically equivalent to separate W_i when dimensions divide evenly.

5.4 Full tiny numerical multi-head example (h=2)

$d_{\text{model}}=4, h=2, d_k=2, T=2$ tokens. Rows = ["I", "cats"] toy vectors.

```
X = [[1, 0, 1, 0],      # I
      [0, 1, 0, 1]]    # cats      (2x4)
```

--- Head 1 projections (toy) ---

```
Q1 = [[1, 0],          K1 = [[1, 0],          V1 = [[1, 0],
      [0, 1]]           [0, 1]]           [0, 1]]
```

```
S1 = Q1 K1^T = [[1, 0],
                [0, 1]]
```

```
scaled1 = S1/√2 ≈ [[0.707, 0],
                  [0, 0.707]]
```

```
A1 ≈ [[0.668, 0.332],
      [0.332, 0.668]]
```

```
head1 = A1 V1 ≈ [[0.668, 0.332],
                 [0.332, 0.668]]
```

--- Head 2 projections (different subspace) ---

```
Q2 = [[0, 1],          K2 = [[0, 1],          V2 = [[0, 1],
      [1, 0]]           [1, 0]]           [1, 0]]
```

```
S2 = Q2 K2^T = [[1, 0],
                [0, 1]]
```

A2 ≈ same structure as A1 (in this toy)

```
head2 ≈ [[0.668, 0.332],
```

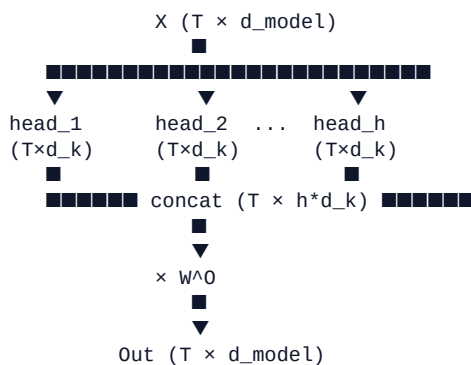
```
[0.332, 0.668]]

--- Concatenate ---
concat[0] = [0.668, 0.332, 0.668, 0.332]
concat[1] = [0.332, 0.668, 0.332, 0.668]      # shape (2×4)

--- Output projection W^O (4×4), toy identity ---
Out = concat @ I = concat

If W^O were not identity, it would mix head channels into final features.
```

5.5 ASCII picture of multi-head



5.6 What fails if multi-head is removed?

Single-head with same compute budget has less ability to represent multiple incompatible attention patterns at once. Ablations in the paper show that reducing heads (while holding compute roughly constant via d_k) hurts quality past a point — diversity of subspaces matters.

Paper connection: §3.2.2 Multi-Head Attention; Figure 2 (right); ablation tables on number of heads.

5.7 Parameter counting (Base, one MHA module)

W_Q, W_K, W_V, W_O each: 512×512 (+ biases optional)
 $\approx 4 * 512 * 512 = 1,048,576$ weights for projections alone per MHA block.

Encoder: 6 layers $\rightarrow$ 6 self-attn MHAs
 Decoder: 6 layers $\rightarrow$ 6 masked self-attn + 6 cross-attn = 12 MHAs
 Attention projections + FFNs dominate Base parameter count.

5.8 Common implementation reshape

```
x: (B, T, 512)
q = W_q(x): (B, T, 512)
q = q.view(B, T, 8, 64).transpose(1, 2)  $\rightarrow$  (B, 8, T, 64)
# batched attention over heads
# reverse to concat: transpose + contiguous view  $\rightarrow$  (B, T, 512)
```

5.9 Heads are not free interpretability

Papers and blogs sometimes label heads (“syntax head”, “anaphora head”). Treat such labels as hypotheses. Architecturally, multi-head guarantees capacity for multiple patterns — not that each head learns a clean linguistic category.

PART 6 — Positional Encoding

6.1 The problem

Self-attention is permutation-equivariant over positions if we only feed token embeddings: shuffling tokens would produce shuffled outputs with the same pairwise scores structure. Word order matters (“dog bites man” ≠ “man bites dog”). Without position information, the model is closer to a bag-of-vectors mixer.

If PE is absent:

Attention(X) for [A,B,C] vs Attention(X) for [C,B,A]
→ outputs are the same vectors permuted – order not represented.

6.2 The 2017 solution: sinusoidal PE added to embeddings

$$\text{PE}(\text{pos}, 2i) = \sin(\text{pos} / 10000^{(2i/d_{\text{model}})})$$

$$\text{PE}(\text{pos}, 2i+1) = \cos(\text{pos} / 10000^{(2i/d_{\text{model}})})$$

- pos = token position (0, 1, 2, ...)
- i = dimension-pair index (0 ... d_model/2 – 1)
- even dims use sin; odd dims use cos
- different frequencies across dimensions → unique patterns per position

6.3 Why add (not concatenate)?

Adding keeps shape (T × d_model) unchanged and injects position into the same space the model already uses. No extra width; residual pathway stays clean. The network learns to use the combined signal. Concatenation would change d_model or require an immediate projection.

6.4 Why sine/cosine?

The paper argues relative positions can be expressed as linear functions of these encodings (useful for attending by offset), and they can extrapolate to sequence lengths longer than those seen in training. Absolute learned position embeddings were also tried and performed similarly on the paper’s tasks — but sinusoids were chosen for the extrapolation argument.

Modern models often use learned absolute, relative, or RoPE positions — different from original 2017. Do not silently replace sinusoids when explaining this paper.

6.5 Worked PE numbers (d_model=4)

d_model=4 → dimension pairs i=0 and i=1

For i=0: $10000^{(2 \cdot 0/4)} = 10000^0 = 1$

For i=1: $10000^{(2 \cdot 1/4)} = 10000^{(0.5)} = 100$

Position 0:

PE(0,0)=sin(0/1)=0

PE(0,1)=cos(0/1)=1

PE(0,2)=sin(0/100)=0

PE(0,3)=cos(0/100)=1

PE[0] = [0, 1, 0, 1]

Position 1:

PE(1,0)=sin(1/1)=sin(1)≈0.8415

PE(1,1)=cos(1/1)=cos(1)≈0.5403

PE(1,2)=sin(1/100)=sin(0.01)≈0.0100

PE(1,3)=cos(1/100)=cos(0.01)≈0.99995

PE[1] ≈ [0.8415, 0.5403, 0.0100, 0.99995]

Position 2:

```

PE(2,0)=sin(2)≈0.9093
PE(2,1)=cos(2)≈-0.4161
PE(2,2)=sin(0.02)≈0.0200
PE(2,3)=cos(0.02)≈0.9998
PE[2] ≈ [0.9093, -0.4161, 0.0200, 0.9998]

```

Notice: low dimensions ($i=0$) change quickly with pos;
high dimensions ($i=1$) change slowly – multi-scale position signal.

6.6 Adding PE to our translation example

```

Source tokens: I, love, cats      positions 0,1,2
Suppose toy embeddings E (3×4):
E = [[0.1, 0.2, 0.3, 0.4],
      [0.5, 0.1, 0.0, 0.2],
      [0.2, 0.4, 0.1, 0.3]]

```

```

X0 = E + PE[0:3]
X0[0] = [0.1, 0.2, 0.3, 0.4] + [0, 1, 0, 1] = [0.1, 1.2, 0.3, 1.4]
X0[1] = [0.5, 0.1, 0.0, 0.2] + [0.84, 0.54, 0.01, 1.00] ≈ [1.34, 0.64, 0.01, 1.20]
X0[2] ≈ [0.2, 0.4, 0.1, 0.3] + [0.91, -0.42, 0.02, 1.00] ≈ [1.11, -0.02, 0.12, 1.30]

```

This X_0 is the encoder stack's input.

6.7 What fails without PE?

Order confusion / bag-of-vectors behavior. The model cannot reliably distinguish “I love cats” from “cats love I”. Architecture would not know sequence order.

Paper connection: §3.5 Positional Encoding; also compared to learned embeddings.

6.8 Relative-position intuition from sinusoids

Because sin/cos of different frequencies behave like a geometric progression, there exist linear transforms relating $PE(pos+k)$ to $PE(pos)$. The paper cites this as a reason the model can learn to attend by relative offset. You do not need the trig proof to use PE — but it explains why random fixed vectors would be a weaker inductive bias.

6.9 Decoder PE is independent

Encoder and decoder each add PE to their own embeddings. Position 0 on the English side is not the same slot as position 0 on the French side; each sequence has its own timeline. Cross-attention aligns them by content, not by forcing equal indices.

```

Source positions: 0:I    1:love  2:cats
Target positions: 0:BOS  1:J'aime 2:les  3:chats

```

Cross-attn may map target pos3 → source pos2 even though indices differ.

6.10 PE frequency table ($d_{\text{model}}=8$, $pos=0..3$)

$i=0$ denom=1; $i=1$ denom=10; $i=2$ denom=100; $i=3$ denom=1000 (approx via $10000^{(2i/8)}$)

pos/dim	0(sin)	1(cos)	2(sin)	3(cos)	4(sin)	5(cos)	...
0	0.000	1.000	0.000	1.000	0.000	1.000	
1	0.841	0.540	0.100	0.995	0.010	1.000	
2	0.909	-0.416	0.199	0.980	0.020	1.000	
3	0.141	-0.990	0.296	0.955	0.030	1.000	

Low dims oscillate fast (fine position); high dims change slowly (coarse position).

PART 7 — The Feed-Forward Network

$$\text{FFN}(x) = \max(0, x W_1 + b_1) W_2 + b_2$$

7.1 Why FFN after attention?

Attention mixes information **across tokens** (communication). The FFN processes each token independently, applying a nonlinear transform — “thinking” content-wise after communication. Attention alone is mostly linear mixing of values (plus softmax nonlinearity on weights); deep nonlinear per-token capacity comes largely from the FFN.

7.2 Expansion and contraction

```
x:      (T × d_model)          e.g., 512
W1:     (d_model × d_ff)       e.g., 512 → 2048
hidden = ReLU(x W1 + b1)      (T × 2048)
W2:     (d_ff × d_model)       2048 → 512
out:    (T × d_model)
```

Same FFN weights shared across positions; applied independently per position.
In Base model, FFN holds a large fraction of parameters (two big matrices).

7.3 Tiny numerical FFN

One token $x = [1.0, -0.5]$, $d_{\text{model}}=2$, $d_{\text{ff}}=4$

```
W1 = [[1, 0, 1, 0],
      [0, 1, 0, 1]]      # (2×4)
b1 = [0, 0, 0, 0]
h_pre = x W1 = [1.0, -0.5, 1.0, -0.5]
h = ReLU(h_pre) = [1.0, 0.0, 1.0, 0.0]
```

```
W2 = [[0.5, 0.0],
      [0.0, 0.5],
      [0.5, 0.0],
      [0.0, 0.5]]      # (4×2)
out = h W2 = [1.0, 0.0]
```

ReLU zeroed the negative channels – nonlinearity that stacked linear maps cannot mimic.

7.4 What fails without FFN / without ReLU?

- Without FFN: layers mostly remix existing features; limited per-token nonlinear computation.
- Without ReLU (or other nonlinearity): stacked linear maps collapse to one linear map; expressive power drops.

Paper connection: §3.3 Position-wise Feed-Forward Networks. Note: applied identically at each position.

7.5 FFN as “key-value memory” intuition (optional)

Some analyses view the expanded ReLU layer as detecting features ($W1$ rows/columns as keys) and writing values ($W2$). You do not need this view to implement the paper, but it explains why $d_{\text{ff}} \gg d_{\text{model}}$ helps: more feature detectors per token after attention mixing.

7.6 Where FFN sits relative to attention

```
Attention: tokens talk to each other      (across-T mix)
FFN:       each token thinks alone         (across-d nonlinear)
```

Stacking them = communicate, then compute, repeatedly.

PART 8 — Residual Connections + Layer Normalization

Deep Transformers need two engineering ideas that are easy to skip when staring at attention: residuals (so layers refine rather than overwrite) and normalization (so scales stay trainable). The 2017 paper's specific order is Post-LN — memorize it.

8.1 Residual connection intuition

Instead of replacing a representation with $\text{Sublayer}(x)$, use $x + \text{Sublayer}(x)$. The sublayer learns a refinement (residual) rather than a full rewrite. This eases gradient flow in deep stacks (N=6 encoder + N=6 decoder is already deep for 2017).

```
output_raw = x + MultiHeadAttention(x)  # same idea for FFN
```

8.2 Layer normalization

LayerNorm normalizes features of each token (across the d_{model} dimension) to stable mean/variance, then applies learned scale γ and bias β . Stabilizes training of deep networks. Unlike BatchNorm, it does not depend on batch statistics — natural for variable-length sequences.

```
For one token vector  $x \in \mathbb{R}^d$ :  
 $\mu = \text{mean}(x)$ ;  $\sigma^2 = \text{variance}(x)$   
 $x_{\text{norm}} = (x - \mu) / \sqrt{\sigma^2 + \epsilon}$   
 $y = \gamma \cdot x_{\text{norm}} + \beta$ 
```

8.3 Original 2017: Post-LN (Add & Norm)

LayerNorm($x + \text{Sublayer}(x)$)

Order: compute sublayer → add residual → LayerNorm. This is **Post-LN**. Figure 1's “Add & Norm” boxes mean exactly this.

```
# Original paper encoder block (Post-LN):  
y = LayerNorm( x + Dropout(SelfAttn(x)) )  
z = LayerNorm( y + Dropout(FFN(y)) )
```

8.4 Modern contrast: Pre-LN

```
# Common modern Pre-LN block:  
y = x + Dropout(SelfAttn(LayerNorm(x)))  
z = y + Dropout(FFN(LayerNorm(y)))  
# often final LayerNorm at the very end of the stack
```

*Pre-LN often trains more stably for very deep stacks and is common in GPT-style models. **Do not silently substitute Pre-LN when explaining the 2017 paper.** If you implement Post-LN vs Pre-LN wrong, you are not reproducing Vaswani et al.*

8.5 Side-by-side

Aspect	Post-LN (2017)	Pre-LN (modern)
Norm position	After residual add	Before sublayer
Paper Fig. 1	Matches	Does not match
Deep training	Can need careful LR	Often stabler deep
Output scale	Normed every block	Residual path un-normed

8.6 What fails without residuals / norms?

- Without residuals: deep stacks become hard to optimize; gradients degrade.
- Without LayerNorm: activation scales drift; training unstable.
- Wrong order (Pre vs Post): may still train, but it is a different architecture than the paper.

PART 9 — Build the Encoder From Scratch

9.1 One encoder layer (2017 Post-LN)

One Encoder Layer:

```
Input X (T × d_model)
■
■ ■ Multi-Head Self-Attention
■   Q,K,V all from X
■
■ ■ Dropout on attention output
■ ■ Add & Norm: LayerNorm(X + Attn)    # Post-LN
■
■ ■ Position-wise FFN (ReLU expand/contract)
■
■ ■ Dropout on FFN output
■ ■ Add & Norm: LayerNorm(Y + FFN)
■
▼
Output (T × d_model)  # same shape as input
```

9.2 Stacking N=6

Base Transformer stacks N=6 identical encoder layers (shared structure, separate parameters). Lower layers often capture more local/surface patterns; higher layers more abstract combinations — not a rigid law, but why depth helps: repeated mix (attention) + transform (FFN).

```
X0 = Embedding(src_ids) * √d_model + PE
X1 = EncoderLayer1(X0)
X2 = EncoderLayer2(X1)
...
X6 = EncoderLayer6(X5)
Memory M = X6      # contextual source for decoder cross-attention
```

Note: the paper scales embeddings by $\sqrt{d_{\text{model}}}$ before adding PE.

9.3 Running example shapes (“I love cats”)

Source $T_{\text{src}} = 3$, $d_{\text{model}} = 512$ (paper) or 8 (toy)

```
ids → Emb (3×512) → *√512 → +PE → X0 (3×512)
6 × EncoderLayer → M (3×512)
```

Each layer preserves (3×512). Only internal Q/K/V widths are 64 per head.

9.4 What the encoder is for

Encoder output is a contextualized sequence of source tokens — memory for the decoder to attend over. It is not yet French words; it is a rich English-side representation.

9.5 Encoder-only vs full Transformer

If you chop off the decoder and add a classification head, you get an encoder-only model family (BERT-like). That is later work. The 2017 paper’s MT system needs both stacks.

9.6 Numerical residual step inside encoder

```
Toy d_model=2, one token:
x = [1.0, 2.0]
attn_out = [0.2, -0.5]
x1_pre = x + attn_out = [1.2, 1.5]
x1 = LayerNorm(x1_pre)  # mean/var normalize then γ,β
```

```
ffn_out = [0.1, 0.1]  
x2 = LayerNorm(x1 + ffn_out)
```

Shape unchanged; content refined. Repeat N times.

PART 10 — The Decoder (Masking + Cross-Attention)

10.1 Why the decoder differs

The decoder must generate the target left-to-right without cheating by looking at future target tokens. It also must read the encoder's source representations. So each decoder layer needs three sublayers: (1) masked self-attention, (2) cross-attention, (3) FFN — each wrapped with Add & Norm (Post-LN).

10.2 Causal masking (look-ahead mask) — detailed

During training, the full target sentence exists in the batch. Without a mask, self-attention would let position i see positions $> i$ — information leakage from the future. The causal mask sets future scores to $-\infty$ before softmax so those weights become 0.

Target: "<BOS> J'aime les chats" (predicting French)

Causal mask M (0=allowed shown as 0, blocked as $-\infty$ before softmax):

	BOS	J'aime	les	chats
BOS	0	$-\infty$	$-\infty$	$-\infty$
J'aime	0	0	$-\infty$	$-\infty$
les	0	0	0	$-\infty$
chats	0	0	0	0

Rule: query position i may only see keys $j \leq i$.

After softmax, each row is a distribution over the allowed prefix only.

10.3 Numerical masked attention (tiny)

$T=3$, $d_k=2$, ignore projections; $Q=K=V$ given:

$Q=K = \begin{bmatrix} [1, 0], [0, 1], [1, 1] \end{bmatrix}$

$S = QK^T = \begin{bmatrix} [1, 0, 1], \\ [0, 1, 1], \\ [1, 1, 2] \end{bmatrix}$

Apply causal mask (set $j>i$ to $-\infty$):

$S_{\text{masked}} = \begin{bmatrix} [1, -\infty, -\infty], \\ [0, 1, -\infty], \\ [1, 1, 2] \end{bmatrix}$

$\text{scaled} = S_{\text{masked}} / \sqrt{2}$

$A = \text{softmax_row}(\text{scaled})$

Row0: only pos0 $\rightarrow A[0] = [1, 0, 0]$

Row1: pos0,1 $\rightarrow \text{softmax}([0, 0.707]) \approx [0.332, 0.668, 0]$

Row2: all three $\rightarrow$ as unmasked row for $[0.707, 0.707, 1.414]$

Without mask, row0 could put weight on future tokens — illegal for autoregressive MT.

10.4 Cross-attention (encoder–decoder attention)

After masked self-attention, each decoder position needs source information. Cross-attention uses:

Q = from decoder states shape $(T_{\text{tgt}} \times d_k)$

K = from encoder memory M shape $(T_{\text{src}} \times d_k)$

V = from encoder memory M shape $(T_{\text{src}} \times d_v)$

Scores: $(T_{\text{tgt}} \times T_{\text{src}})$ — each French position looks over English positions.

Running example alignments (illustrative):

"J'aime" attends strongly to "love" (and maybe "I")

"les" may attend to function-word structure

"chats" attends strongly to "cats"

10.5 One decoder layer ASCII

```
Y_in (T_tgt × d_model)
■
■ ■ Masked Multi-Head Self-Attention (Q=K=V=Y)
■ ■ Add & Norm (Post-LN)
■
■ ■ Multi-Head Cross-Attention (Q=Y, K=V=Memory)
■ ■ Add & Norm
■
■ ■ FFN
■ ■ Add & Norm
■
▼
Y_out (T_tgt × d_model)
```

10.6 Training decoder input (teacher forcing, shifted right)

```
Target gold:      J'aime  les  chats  <EOS>
Decoder input:    <BOS>   J'aime les  chats
Predict:          J'aime  les  chats  <EOS>
```

Position t 's prediction is trained against gold token t , while only attending to decoder inputs $\leq t$ (via mask).

10.7 What fails if masking / cross-attention is removed?

- **No causal mask:** model cheats during training; inference (no future tokens) mismatches train — poor generation.
- **No cross-attention:** decoder never reads source; cannot translate, only babble target language models.
- **No masked self-attention:** no target-side language modeling / fluency conditioning.

Paper connection: §3.1 decoder; §3.2.3 “Applications of Attention” (self / masked / encoder-decoder).

10.8 Three attention types in one diagram

```
ENCODER self-attention:
  Q,K,V from source; full (unmasked) T_src × T_src

DECODER masked self-attention:
  Q,K,V from target; causal T_tgt × T_tgt

DECODER cross-attention:
  Q from target; K,V from encoder memory; T_tgt × T_src
```

All three are the SAME Attention(Q,K,V) function with different inputs/masks.

10.9 Padding masks (practical detail)

Batches pad short sentences to a common length. Padding positions should not be attended to: set their key scores to $-\infty$ as well (separate from causal masking). Causal mask $\wedge$ padding mask are both applied before softmax in real systems.

```
Example src batch (pad=0):
[I, love, cats, PAD]
mask_key = [1,1,1,0] → column PAD blocked for all queries
```

10.10 Worked cross-attention scores (toy)

$T_{tgt}=2$ ("J'aime", "chats"), $T_{src}=3$ ("I", "love", "cats"), $d_k=2$

```
Q_dec = [[1,0],      # J'aime
          [0,1]]      # chats
K_enc = [[1,0],      # I
          [1,1],      # love
          [0,1]]      # cats
V_enc = [[0,0], [1,0], [0,1]]
```

```

S = Q KT = [[1,1,0],
             [0,1,1]]
A ≈ softmax(S/√2):
  J'aime row peaks on I/love
  chats  row peaks on love/cats

```

Out = A V mixes encoder values into decoder positions.

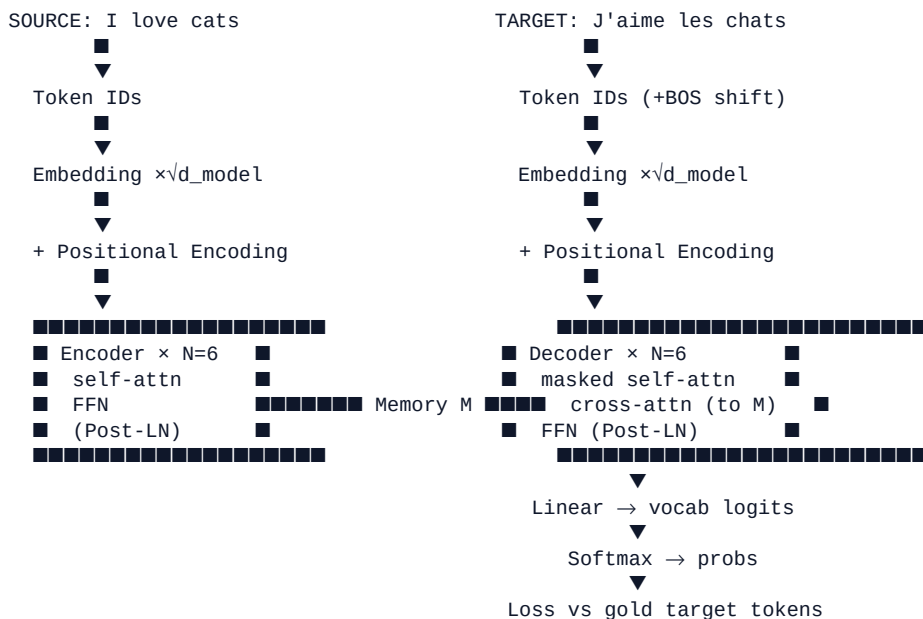
10.11 Layer stack reminder

N=6 means this three-sublayer recipe repeats six times. Early decoder layers may focus more on local target syntax; later layers often strengthen cross-lingual alignment — again a tendency, not a law.

PART 11 — Complete Transformer Data Flow

We now mentally simulate the full system for: “I love cats” → “J’aime les chats”.

11.1 End-to-end ASCII



11.2 Shape trace (toy $d_{\text{model}}=8$, paper note in comments)

Assume $d_{\text{model}}=8$, $h=2$, $d_k=4$, $d_{\text{ff}}=32$, $V_{\text{vocab}}=1000$ (paper: 512/8/64/2048/~32k)

- 1) Source tokenize: ["I", "love", "cats"] → ids (3,)
- 2) Emb_s: (3×8); scale $\sqrt{8}$; +PE → X_0 (3×8)
- 3) Encoder layer (repeat 6×):
 - Q, K, V projections → heads → concat → (3×8)
 - Add&Norm → FFN → Add&Norm → still (3×8)
 - Memory M: (3×8)
- 4) Decoder inputs (teacher forcing):
["<BOS>", "J'aime", "les", "chats"] → Y_0 (4×8) after emb+PE
- 5) Decoder layer (repeat 6×):
 - a) masked self-attn on Y: scores (4×4) with causal mask → (4×8)
 - b) cross-attn: Q from Y (4× d_k), K/V from M (3× d_k)
scores (4×3) → output (4×8)
 - c) FFN → (4×8)
- 6) Final linear: (4×8) @ $W_{\text{vocab}}(8 \times 1000)$ → logits (4×1000)
- 7) Softmax over vocab dim → probs (4×1000)
- 8) Cross-entropy vs gold ["J'aime", "les", "chats", "<EOS>"]

Mental rule: every block preserves $(T \times d_{\text{model}})$ until the final vocab projection.

11.3 Token-level story (one French word)

When the decoder is at the position that should predict “chats”:

- Masked self-attn lets it read <BOS>, J’aime, les (not future).
- Cross-attn lets it look at contextualized “I”, “love”, “cats” — ideally heavy on “cats”.
- FFN nonlinearly transforms that mixture.
- Final linear+softmax puts high probability on “chats”.

11.4 Training vs inference dataflow difference

TRAINING:

full target available → parallel teacher forcing over all tgt positions
causal mask prevents cheating
one forward pass over whole target length

INFERENCE:

encode source once → M
generate y1, then y2, ... sequentially
each step runs decoder on prefix generated so far
stop at <EOS> or max length

Same architecture weights; different iteration pattern over the decoder.

11.5 Full tensor diary (write this on paper once)

Let d=512, h=8, d_k=64, N=6, V=32000

```
src_ids:      (B, 3)
src_emb:      (B, 3, 512)
src_x:        (B, 3, 512)  # *sqrt(d)+PE
after enc:    (B, 3, 512)  # Memory
```

```
tgt_ids_in:   (B, 4)
tgt_x:        (B, 4, 512)
after dec:    (B, 4, 512)
logits:       (B, 4, V)
loss:         scalar
```

Inside one MHA:

```
qkv: (B,4,512) → split (B,8,4,64)
scores self: (B,8,4,4) or cross (B,8,4,3)
```

11.6 Where information moves

- Encoder self-attn: English ↔ English
- Decoder masked self-attn: French prefix ↔ French prefix
- Cross-attn: French ↔ English
- FFN: within-token feature remix (no inter-token mix)
- Final linear: features → vocabulary evidence

11.7 Failure modes mapped to missing pieces

Symptoms → likely missing piece

```
-----
Ignores word order           → PE
Can't resolve "it"           → self-attention depth/capacity
Translates without source    → cross-attention
Copies future target words   → causal mask (train/test mismatch)
Deep net won't train         → residuals / LayerNorm (and LR schedule)
Fluent but wrong meaning     → weak cross-attn / undertrained encoder
```

PART 12 — Training the Transformer

12.1 Teacher forcing and shifted targets

- **Teacher forcing:** feed the correct previous target tokens as decoder input (not the model's own previous guesses).
- **Shifted inputs:** decoder sees tokens up to $t-1$ when predicting token t .
- **Why:** enables parallel training over time; stabilizes early learning.

Example batch item:

```
src: I love cats
tgt_in: <BOS> J'aime les chats
tgt_out: J'aime les chats <EOS>
```

12.2 Logits, softmax, cross-entropy

At one position, vocab {A,B,C}:

```
logits = [2.0, 0.5, 0.1]
probs = softmax(logits) ≈ [0.659, 0.242, 0.099]
true = A → loss = -log(0.659) ≈ 0.417
```

Full sequence loss = average $-\log p(\text{true}_t)$ over non-pad positions.

12.3 Label smoothing (paper detail)

Instead of hard 1-hot targets, the paper uses label smoothing with $\epsilon_{ls}=0.1$: the correct class gets $1-\epsilon$, and the remaining ϵ is spread over other classes. This penalizes overconfidence and improved BLEU in their experiments. It is a training procedure detail, not an architecture change.

12.4 Optimizer and learning-rate schedule

Adam with $\beta_1=0.9$, $\beta_2=0.98$, $\epsilon=10^{-9}$. Learning rate follows a formula with warmup: increase linearly for warmup_steps (4000), then decay proportionally to $\text{step}^{-0.5}$. This schedule is important to reproduce paper results — architecture alone is not enough.

```
lr_rate = d_model^(-0.5) * min(step^(-0.5), step * warmup^(-1.5))
```

12.5 Regularization

- Dropout $P_{\text{drop}}=0.1$ on attention weights / residual paths / embeddings as specified
- Residual dropout after each sublayer before Add & Norm

12.6 What gradients flow through

Gradients flow through softmax, final linear, decoder stack (including attention weights and $Q/K/V$), cross-attention into encoder memory, encoder stack, embeddings. Attention's softmax and matmuls are fully differentiable — that is why attention is trainable.

12.7 Mini training step checklist

1. Embed+PE source and shifted target
2. Encode source $\rightarrow M$
3. Decode with causal mask + cross-attn to $M \rightarrow \text{logits}$
4. Compute CE (optionally label-smoothed) vs gold next tokens
5. backward(); Adam step with scheduled LR
6. repeat

12.8 Worked micro-batch loss

Suppose $V=4$ tokens {BOS, J'aime, les, chats}, ignore real vocab size.

```
At position predicting "les":
logits = [0.1, 0.2, 2.5, 0.3]    # classes 0..3
p(les)=softmax  $\approx$  0.80
CE =  $-\log(0.80) \approx 0.223$ 
```

```
At position predicting "cats":
logits = [0.0, 0.1, 0.2, 1.5]
p  $\approx$  0.55  $\rightarrow$  CE  $\approx$  0.598
```

Mean loss $\approx$ 0.41 for these two positions.
Backprop increases logits of correct classes on the next update.

12.9 What training is NOT

- Not searching attention patterns by hand — gradients discover them
- Not requiring force-aligned word dictionaries (attention learns soft alignment)
- Not identical to inference compute graph over time (teacher forcing vs autoregressive)

12.10 Gradient intuition through attention

If the model assigned too little weight to “cats” when predicting “cats”, the loss gradient encourages increasing the score $Q_cats \cdot K_cats$ (via W_Q/W_K) and/or adjusting V_cats and W_O . You rarely debug this by hand — but knowing the pathway demystifies “attention learns alignments.”

PART 13 — Inference / Generation

13.1 Autoregressive loop

Encode source once → Memory M

```
y = [<BOS>]
loop:
  decoder(y, M) with causal mask
  take logits at last position
  choose next token:
    greedy: argmax
    or sampling / beam search
  append token to y
  stop at <EOS> or max length
```

Example:

```
y = [<BOS>]
→ predicts J'aime
y = [<BOS>, J'aime]
→ predicts les
y = [<BOS>, J'aime, les]
→ predicts chats
y = [<BOS>, J'aime, les, chats]
→ predicts <EOS> → stop
```

13.2 Training vs inference

Training uses teacher forcing (parallel over target length). Inference is sequential token-by-token — a major speed cost of autoregressive decoding. Exposure bias: train sees gold prefixes; inference sees its own prefixes — a known issue.

13.3 Greedy vs beam search

- **Greedy:** always pick top-1; fast; can commit early mistakes.
- **Beam search:** keep top-B partial hypotheses; often better BLEU; slower; paper reports beam results.

Paper connection: §5–6 mention beam search settings used for BLEU numbers.

13.4 Step-by-step greedy decode for the running example

M = Encode("I love cats")

```
step1: y=[BOS]           → probs → pick J'aime
step2: y=[BOS,J'aime]     → probs → pick les
step3: y=[BOS,J'aime,les] → probs → pick chats
step4: y=[...,chats]      → probs → pick EOS → stop
```

At each step, masked self-attn only sees the growing prefix;
cross-attn always sees full M.

13.5 Why caching matters (implementation note)

Naive inference recomputes attention for the whole prefix every step. Production systems cache past K/V (KV cache). Conceptually identical results; engineering speedup. The 2017 paper does not require you to invent KV caching to understand the model.

PART 14 — The Paper's Equations (Decoded)

14.1 Scaled Dot-Product Attention

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}(\mathbf{Q}\mathbf{K}^T / \sqrt{d_k})\mathbf{V}$$

Symbols/shapes/intuition/computation: Part 3. Location: core of every attention head. Why: content-based soft lookup that is GPU-friendly (matmul).

14.2 Multi-Head

$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_i)W^O \text{ with } \text{head}_i = \text{Attention}(\mathbf{Q}W_i^Q, \mathbf{K}W_i^K, \mathbf{V}W_i^V)$$

Location: encoder self-attn, decoder masked self-attn, decoder cross-attn. Why: multiple subspaces / relation types.

14.3 FFN

$$\text{FFN}(\mathbf{x}) = \max(0, \mathbf{x}W_1 + b_1)W_2 + b_2$$

Location: after attention sublayer in every encoder/decoder layer. Why: per-position nonlinear capacity.

14.4 Positional Encoding

$$\text{PE}(\text{pos}, 2i) = \sin(\text{pos} / 10000^{(2i/d_{\text{model}})}); \text{PE}(\text{pos}, 2i+1) = \cos(\dots)$$

Location: added to token embeddings at encoder and decoder bottoms. Why: inject order.

14.5 Add & Norm (Post-LN)

$$\text{LayerNorm}(\mathbf{x} + \text{Sublayer}(\mathbf{x}))$$

Location: around every sublayer in Fig. 1. Why: deep residual training stability. Contrast: modern Pre-LN is $\text{Sublayer}(\text{LayerNorm}(\mathbf{x})) + \mathbf{x}$.

14.6 Equation → module map

Equation	Module	Course part
Attention(...)	ScaledDotProductAttention	Part 3
MultiHead(...)	MultiHeadAttention	Part 5
FFN(...)	Positionwise FFN	Part 7
PE(...)	Sinusoidal PE	Part 6
LayerNorm(x+...)	AddNorm Post-LN	Part 8

PART 15 — Original 2017 Architecture Specs

15.1 ARCHITECTURE (what the machine is)

- $N = 6$ encoder layers, $N = 6$ decoder layers
- $d_{\text{model}} = 512$ (Base); Big uses 1024
- $h = 8$ heads (Base) $\rightarrow d_k = d_v = 64$
- $d_{\text{ff}} = 2048$ (Base)
- sinusoidal positional encoding
- learned embeddings; residual + LayerNorm in **Post-LN** order
- shared embedding weights with pre-softmax linear (weight tying)
- no recurrence, no convolution in the backbone

15.2 TRAINING PROCEDURE

- WMT 2014 EN-DE and EN-FR translation
- next-token cross-entropy with teacher forcing
- Adam optimizer with custom LR schedule (warmup then decay)
- label smoothing $\epsilon_{\text{ls}} = 0.1$

15.3 IMPLEMENTATION DETAILS

- dropout $P_{\text{drop}} = 0.1$
- batching by approximate token counts
- beam search at inference for reported BLEU
- checkpoint averaging sometimes used for reporting

Keep these three buckets separate when discussing the paper.

15.4 Base vs Big (high level)

Base: $d_{\text{model}}=512$, $h=8$, $d_{\text{ff}}=2048$, $\text{dropout}=0.1$

Big: $d_{\text{model}}=1024$, $h=16$, $d_{\text{ff}}=4096$, $\text{dropout}=0.3$

Bigger models $\rightarrow$ better BLEU, more compute.

PART 16 — Complexity and Why Transformers Scale

Per layer (conceptual) – paper Table 1 spirit:

	Complexity/layer	Sequential ops	Max path length
Self-Attention	$O(T^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent	$O(T \cdot d^2)$	$O(T)$	$O(T)$
Convolution	$O(k \cdot T \cdot d^2)$	$O(1)$	$O(\log_k T)$ or $O(T/k)$

T =sequence length, d =dimension, k =kernel size.

Transformers won despite $O(T^2)$ because: excellent GPU parallelization, short path lengths for long-range dependencies, and favorable scaling with data/compute. Quadratic cost becomes painful for very long context — motivating later efficient-attention variants (sparse, linear, FlashAttention engineering, etc.).

Key insight: sequential operations (RNN depth in time) hurt wall-clock training more than FLOPs alone suggest.

PART 17 — Reading the Results Section

- **WMT:** standard machine translation benchmarks (news translation).
- **BLEU:** n-gram overlap metric vs references; higher is better (imperfect but standard then).
- **Baselines:** prior RNN/CNN translation systems; compare quality and training cost.
- **Parameter counts / training time:** not only accuracy — efficiency claims matter.
- **Ablation:** remove or alter a piece (heads, PE type, d_k , ...) and measure drop — shows which parts matter.

Demonstrated $\neq$ suggested. Demonstrated: numbers on stated tasks. Suggested: broader speculation (e.g., applicability beyond MT). Read tables carefully before believing headlines.

When you reread §6: ask “what exactly was measured?” and “what was held constant in the ablation?”

PART 18 — Limitations of the Original Transformer

- Quadratic attention memory/compute in T
- Long-context difficulty at original design point
- Autoregressive decoding latency at inference
- Absolute sinusoidal PE is not the end of position modeling
- Needs large data/compute to shine
- Attention weights $\neq$ complete interpretability
- Teacher forcing / exposure bias

Later differences (do not confuse with 2017):

- Pre-LN instead of Post-LN
- Learned / relative / RoPE positions
- Decoder-only GPT-style LMs; encoder-only BERT
- Efficient / approximate attention; better kernels
- Huge scale, instruction tuning, RLHF — far beyond original MT setting

Know these exist so you don't confuse them with Vaswani 2017 when reconstructing Fig. 1.

PART 19 — Implement a Miniature Transformer (PyTorch)

*Below is a compact educational implementation. Every class maps to a paper component. Shapes assume batch first: (B, T, d). This uses **Post-LN** to match 2017.*

19.1 Scaled dot-product + multi-head

```
import math
import torch
import torch.nn as nn
import torch.nn.functional as F

class ScaledDotProductAttention(nn.Module):
    def forward(self, Q, K, V, mask=None):
        # Q,K,V: (B, h, T, d_k)
        d_k = Q.size(-1)
        scores = torch.matmul(Q, K.transpose(-2, -1)) / math.sqrt(d_k)
        if mask is not None:
            scores = scores.masked_fill(mask == 0, -1e9)
        attn = F.softmax(scores, dim=-1) # (B,h,T_q,T_k)
        return torch.matmul(attn, V), attn # (B,h,T_q,d_k)

class MultiHeadAttention(nn.Module):
    def __init__(self, d_model, h):
        super().__init__()
        assert d_model % h == 0
        self.d_k = d_model // h
        self.h = h
        self.W_q = nn.Linear(d_model, d_model)
        self.W_k = nn.Linear(d_model, d_model)
        self.W_v = nn.Linear(d_model, d_model)
        self.W_o = nn.Linear(d_model, d_model)
        self.attn = ScaledDotProductAttention()

    def split_heads(self, x):
        # (B,T,d) -> (B,h,T,d_k)
        B, T, _ = x.shape
        return x.view(B, T, self.h, self.d_k).transpose(1, 2)

    def forward(self, query, key, value, mask=None):
        Q = self.split_heads(self.W_q(query))
        K = self.split_heads(self.W_k(key))
        V = self.split_heads(self.W_v(value))
        if mask is not None:
            mask = mask.unsqueeze(1) # broadcast over heads
        out, weights = self.attn(Q, K, V, mask)
        out = out.transpose(1, 2).contiguous().view(
            query.size(0), query.size(1), -1)
        return self.W_o(out), weights
```

19.2 FFN, Post-LN AddNorm, sinusoidal PE

```
class PositionwiseFFN(nn.Module):
    def __init__(self, d_model, d_ff):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(d_model, d_ff), nn.ReLU(), nn.Linear(d_ff, d_model)
        )

    def forward(self, x):
        return self.net(x)

class AddNorm(nn.Module):
    # Post-LN as in 2017 paper: LayerNorm(x + Dropout(sublayer(x)))
    def __init__(self, d_model, dropout=0.1):
        super().__init__()
        self.norm = nn.LayerNorm(d_model)
        self.drop = nn.Dropout(dropout)
```

```

def forward(self, x, sublayer_out):
    return self.norm(x + self.drop(sublayer_out))

class SinusoidalPE(nn.Module):
    def __init__(self, d_model, max_len=5000):
        super().__init__()
        pe = torch.zeros(max_len, d_model)
        pos = torch.arange(0, max_len).unsqueeze(1).float()
        div = torch.exp(torch.arange(0, d_model, 2).float() *
                        (-math.log(10000.0) / d_model))
        pe[:, 0::2] = torch.sin(pos * div)
        pe[:, 1::2] = torch.cos(pos * div)
        self.register_buffer("pe", pe.unsqueeze(0)) # (1,L,d)
    def forward(self, x):
        return x + self.pe[:, :x.size(1)]

```

19.3 Encoder and decoder layers

```

class EncoderLayer(nn.Module):
    def __init__(self, d_model, h, d_ff, dropout=0.1):
        super().__init__()
        self.self_attn = MultiHeadAttention(d_model, h)
        self.ffn = PositionwiseFFN(d_model, d_ff)
        self.an1 = AddNorm(d_model, dropout)
        self.an2 = AddNorm(d_model, dropout)
    def forward(self, x, src_mask=None):
        a, _ = self.self_attn(x, x, x, src_mask)
        x = self.an1(x, a)
        return self.an2(x, self.ffn(x))

class DecoderLayer(nn.Module):
    def __init__(self, d_model, h, d_ff, dropout=0.1):
        super().__init__()
        self.self_attn = MultiHeadAttention(d_model, h)
        self.cross_attn = MultiHeadAttention(d_model, h)
        self.ffn = PositionwiseFFN(d_model, d_ff)
        self.an1 = AddNorm(d_model, dropout)
        self.an2 = AddNorm(d_model, dropout)
        self.an3 = AddNorm(d_model, dropout)
    def forward(self, y, memory, tgt_mask=None, memory_mask=None):
        a, _ = self.self_attn(y, y, y, tgt_mask)
        y = self.an1(y, a)
        c, _ = self.cross_attn(y, memory, memory, memory_mask)
        y = self.an2(y, c)
        return self.an3(y, self.ffn(y))

def causal_mask(T, device):
    # 1 = keep, 0 = block
    return torch.tril(torch.ones(1, T, T, device=device))

```

19.4 Full miniature model skeleton

```

class MiniTransformer(nn.Module):
    def __init__(self, src_vocab, tgt_vocab, d_model=64, h=4,
                 d_ff=256, N=2, dropout=0.1):
        super().__init__()
        self.d_model = d_model
        self.src_emb = nn.Embedding(src_vocab, d_model)
        self.tgt_emb = nn.Embedding(tgt_vocab, d_model)
        self.pe = SinusoidalPE(d_model)
        self.enc = nn.ModuleList(
            [EncoderLayer(d_model, h, d_ff, dropout) for _ in range(N)])
        self.dec = nn.ModuleList(
            [DecoderLayer(d_model, h, d_ff, dropout) for _ in range(N)])
        self.out = nn.Linear(d_model, tgt_vocab)

    def encode(self, src, src_mask=None):
        x = self.pe(self.src_emb(src) * math.sqrt(self.d_model))
        for layer in self.enc:
            x = layer(x, src_mask)

```

```

    return x

    def decode(self, tgt, memory, tgt_mask=None, memory_mask=None):
        y = self.pe(self.tgt_emb(tgt) * math.sqrt(self.d_model))
        for layer in self.dec:
            y = layer(y, memory, tgt_mask, memory_mask)
        return self.out(y)

    def forward(self, src, tgt, src_mask=None, tgt_mask=None):
        memory = self.encode(src, src_mask)
        return self.decode(tgt, memory, tgt_mask, src_mask)

# Train: logits = model(src, tgt_in, tgt_mask=causal_mask(T))
#         loss = cross_entropy(logits.reshape(-1,V), tgt_out.reshape(-1))
# Infer: encode once; append argmax tokens until EOS

```

19.5 Line-to-equation map

- $\text{matmul } Q \ K^T \rightarrow \text{scores}; / \sqrt{\text{scale}} \rightarrow \text{scale}; \text{masked_fill} \rightarrow \text{causal/pad}; \text{softmax} \rightarrow \text{weights}; \text{matmul } V \rightarrow \text{output}$
- $\text{view/transpose} \rightarrow \text{multi-head reshape}; W_o \rightarrow W^O$
- $\text{LayerNorm}(x+\dots) \rightarrow \text{Add \& Norm Post-LN}$
- $\sin/\cos \text{ buffer} \rightarrow \text{sinusoidal PE}; \sqrt{\text{d_model}} \rightarrow \text{paper embedding scale}$

Exercise: change AddNorm to Pre-LN and observe that you have left the 2017 architecture.

19.6 Tiny train / infer sketch

```

model = MiniTransformer(src_vocab=50, tgt_vocab=50, d_model=64, h=4, N=2)
opt = torch.optim.Adam(model.parameters(), lr=1e-3, betas=(0.9,0.98), eps=1e-9)

# --- one training step ---
src = torch.tensor([[1,2,3]])          # I love cats
tgt_in = torch.tensor([[4,5,6,7]])     # BOS J'aime les chats
tgt_out = torch.tensor([[5,6,7,8]])     # J'aime les chats EOS
T = tgt_in.size(1)
mask = causal_mask(T, src.device)
logits = model(src, tgt_in, tgt_mask=mask) # (1,4,50)
loss = F.cross_entropy(logits.reshape(-1,50), tgt_out.reshape(-1))
loss.backward(); opt.step(); opt.zero_grad()

# --- greedy inference ---
memory = model.encode(src)
ys = torch.tensor([[4]]) # BOS
for _ in range(10):
    m = causal_mask(ys.size(1), src.device)
    logit = model.decode(ys, memory, tgt_mask=m)
    next_id = logit[:, -1, :].argmax(-1, keepdim=True)
    ys = torch.cat([ys, next_id], dim=1)
    if next_id.item() == 8: # EOS
        break

```

19.7 Debugging shapes (common errors)

- Forgot transpose for scores $\rightarrow$ matmul shape error on last two dims
- Causal mask wrong broadcast $\rightarrow$ attending to future (train/test mismatch)
- Cross-attn called as self-attn ($K=V=\text{decoder}$) $\rightarrow$ no source reading
- Pre-LN accidentally $\rightarrow$ still runs, but not paper-faithful
- Forgot $\sqrt{\text{d_model}}$ on embeddings $\rightarrow$ scale mismatch with PE

19.8 Mapping classes $\rightarrow$ paper figure boxes

```

SinusoidalPE + Embedding     $\rightarrow$  bottom of Fig.1
EncoderLayer.self_attn       $\rightarrow$  encoder Multi-Head Attention
EncoderLayer.ffn             $\rightarrow$  Feed Forward
AddNorm                      $\rightarrow$  Add & Norm (Post-LN)

```

<code>DecoderLayer.self_attn</code>	→ masked Multi-Head Attention
<code>DecoderLayer.cross_attn</code>	→ encoder-decoder Multi-Head Attention
<code>MiniTransformer.out</code>	→ final Linear (+ softmax outside / in loss)

SYNTHESIS — Putting the Whole Machine Together

Before the quiz, lock the global story. The Transformer is not a pile of unrelated tricks; each piece repairs a specific failure mode of “just multiply embeddings.”

S.1 One-page architecture contract

INPUT: token ids (source) and token ids (target, shifted)
REPRESENT: Embedding * $\sqrt{d_{\text{model}}}$ + sinusoidal PE
ENCODE: $N \times (\text{SelfAttn} + \text{PostLN} + \text{FFN} + \text{PostLN})$
DECODE: $N \times (\text{MaskedSelfAttn} + \text{PostLN} + \text{CrossAttn} + \text{PostLN} + \text{FFN} + \text{PostLN})$
PREDICT: Linear to $|V|$; softmax in loss / decoding
TRAIN: teacher forcing + causal mask + CE (+ label smoothing) + Adam schedule
INFER: encode once; autoregressive decode (greedy/beam)

S.2 Running example end-to-end checklist

- English “I love cats” becomes contextual memory M via encoder self-attention.
- French generation starts at <BOS>; masked self-attn keeps it causal.
- Cross-attn grounds “chats” in “cats” (and neighbors).
- Without PE, order collapses; without mask, training cheats; without cross-attn, no translation.

S.3 Post-LN loyalty check

If your mental diagram shows LayerNorm before attention inside the residual branch as the default, you are thinking of modern Pre-LN. For Vaswani 2017, rewrite it as Add then Norm.

S.4 Equation → failure mode table

If removed	Failure
$\sqrt{d_k}$ scale	Softmax saturation / weak grads
Multi-head	One pattern must do all jobs
PE	Bag-of-tokens mixer
FFN nonlinearity	Depth collapses toward linear
Residual	Deep optimization failure
Causal mask	Train/infer mismatch
Cross-attn	No source conditioning

PART 20 — Final Master Understanding (Quiz)

Attempt answers yourself before checking the answer key. Write shapes when relevant.

Questions

Level 1: What is attention, in one precise sentence? What tensors are Q, K, V?

Level 2: Why do we need separate Q, K, and V instead of using X three times without projections?

Level 3: Why scale QK^T by $\sqrt{d_k}$? What goes wrong for large d_k without scaling?

Level 4: Why multi-head attention? What are the shapes for Base ($d_{\text{model}}=512$, $h=8$)?

Level 5: Why positional encoding? Compute $PE(\text{pos}=1, \text{dim}=0)$ and $PE(\text{pos}=1, \text{dim}=1)$ for $d_{\text{model}}=4$.

Level 6: Why residual connections? Write Post-LN vs Pre-LN formulas and say which is 2017.

Level 7: Why masked self-attention? Draw the 4×4 causal mask for “<BOS> J’aime les chats”.

Level 8: How does cross-attention work for translating “cats” → “chats”? Who provides Q vs K/V?

Level 9: Trace one source token and one target position through the full Transformer (embeddings → encoder → decoder → logits), listing shapes with $d_{\text{model}}=512$, $T_{\text{src}}=3$, $T_{\text{tgt}}=4$.

Level 10: Design a Transformer variant for a new problem (e.g., long-document QA or decoder-only LM). What would you change and why? What would you keep?

Answer key (use only after attempting)

A1: Attention is a soft weighted lookup: compare each query to all keys (scaled dots), softmax to weights, mix values. Q asks; K is address; V is content. Shapes: Q ($T_q \times d_k$), K ($T_k \times d_k$), V ($T_k \times d_v$), output ($T_q \times d_v$).

A2: Learned projections let matching (Q/K space) differ from content (V space) and differ across heads. Using raw X without $W_Q/W_K/W_V$ removes the ability to specialize subspaces.

A3: Dot products grow in magnitude with $d_k \rightarrow$ softmax saturates toward one-hot $\rightarrow$ tiny gradients. Dividing by $\sqrt{d_k}$ keeps score variance $\sim O(1)$.

A4: Multiple subspaces/relation types in parallel; concat + W^O mixes them. Base: $d_k=d_v=64$; each head ($T \times 64$); concat ($T \times 512$); out ($T \times 512$).

A5: Without PE, attention is order-agnostic (permutation equivariant). For $d_{\text{model}}=4$, $i=0$ $\text{denom}=1$: $\text{PE}(1,0)=\sin(1)\approx 0.8415$; $\text{PE}(1,1)=\cos(1)\approx 0.5403$.

A6: Residuals ease deep optimization (learn refinements). 2017 Post-LN: $\text{LayerNorm}(x + \text{Sublayer}(x))$. Pre-LN: $x + \text{Sublayer}(\text{LayerNorm}(x))$.

A7: Prevents attending to future target tokens so training matches autoregressive generation. Lower-triangular allowed region; strict upper triangle is $-\infty$ before softmax.

A8: Decoder queries attend to encoder keys/values. For “chats”, Q comes from decoder state at that position; K/V from contextualized source — ideally high weight on “cats”.

A9: $\text{ids} \rightarrow \text{Emb} \times \sqrt{512} + \text{PE} \rightarrow (3 \times 512)$ encoder self-attn/FFN $\times 6 \rightarrow M$ (3×512); tgt (4×512) masked SA $\rightarrow$ cross-attn to M (scores 4×3) $\rightarrow$ FFN $\times 6 \rightarrow$ Linear to $|V| \rightarrow$ softmax. Loss on each of 4 positions.

A10 (example): Decoder-only LM: drop encoder & cross-attn; keep masked self-attn+FFN+PE. Long-doc QA: keep encoder-decoder or encoder-only with span head; maybe sparse attention for long T; justify each change against quadratic cost / task structure. Keep residual+norm+attention core.

Bonus drills (Part 20 continued)

- **B1:** Recompute Part 3’s A V with a different V; show only the readout changes.
- **B2:** Write causal masks for $T=5$ by hand.
- **B3:** For $d_{\text{model}}=512$, $h=8$, list every matrix multiply in one encoder layer.
- **B4:** Explain why teacher forcing needs a mask even though gold targets are available.
- **B5:** Contrast Post-LN vs Pre-LN on a napkin diagram of one block.

Bonus short answers

- **B1:** Scores/weights unchanged if Q,K fixed; output rows change with V.
- **B2:** Lower-triangular ones / upper $-\infty$.
- **B3:** Q,K,V projections; QK^T ; AV; W_O ; FFN W_1 ; FFN W_2 (plus norms).
- **B4:** Gold future tokens exist in the tensor — mask stops leakage so train matches generate.
- **B5:** Post: sublayer $\rightarrow$ add $\rightarrow$ norm; Pre: norm $\rightarrow$ sublayer $\rightarrow$ add.

Final mental model (inevitable architecture)

Need seq2seq $\rightarrow$ encoder/decoder
Need parallel long-range mixing $\rightarrow$ self-attention
Need multi relations $\rightarrow$ multi-head
Need order $\rightarrow$ positional encoding
Need per-token nonlinear think $\rightarrow$ FFN
Need deep stacks to train $\rightarrow$ residual + norm (Post-LN in 2017)
Need left-to-right generation $\rightarrow$ causal mask

Need source info in decoder → cross-attention
Need probabilities → linear + softmax

If someone shows you Fig.1 or $\text{Attention}(Q, K, V) = \text{softmax}(QK^T/\sqrt{d_k})V$,
you should now rebuild the system from first principles.

You are ready to reread Vaswani et al. 2017 — this time as an architect, not as a tourist.